

ASSESSMENT TOOL 1 – INTERACTIVE TREE SIMULATOR

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO3 – Precision (BL3, BL5)	Assessment:	Assessment Tool 1 – Interactive Tree Simulator 100
Weightage:	30% of CIA	Total Marks:	
CO3 Statement:	<i>Solve problems for optimized data storage and retrieval using Binary Search Trees, AVL Trees, BTrees, Red-Black Trees, and Splay Trees.</i>		
Student Name:	N.Adi Narayana	Reg. No.:	192525346
Section / Batch:	AIML	Date:	27/07/2026

Code of Conduct

I_Koppala Navya(192511231) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: Koppala Navya

Instructions

- This assessment tool contains 5 Interactive Tree Simulator questions, Total: 100 Marks.
- Students can use any online/offline Interactive Tree Simulator. Demonstrate insertion, deletion, searching, and traversal operations wherever applicable.
- When a student answer using simulator, in evaluation the answer should have captured screenshots after every major operation.
- Submit the report in PDF format with properly labelled screenshots as proof of completion.

Section / Topic	Focus	Q Nos	Marks
Binary Search Tree	Properties, binary tree and binary search tree, insertion and deletion	1	20
Tree Traversals	Traversal types, In order, Post order and Pre order	2	20
AVL Tree	Balancing factor, Rotation, Types of rotation, examples	3	20
B Tree, 2-3 tree, 2-3-4 tree	Properties, structures and Operations	4	20
Red Black Tree	Properties, Example and Operations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Interactive Tree Simulator

1. Construct a Binary Search Tree (BST) by inserting the following keys: 40, 20, 10, 30, 60, 50, 70
Perform:
 - a. Inorder Traversal
 - b. Search for 70
 - c. Delete 50
 - d. Display the final BST.
2. Complete Tree Traversals by Inserting 55,35,75,25,45,65,85. Find all the traversals. a. Inorder b. Preorder
c. Postorder

3. Insert 40, 20, 60, 10, 30, 50, 70, 5 into an AVL Tree. Determine the balance factor of every node before and after balancing.
4. Construct a B-Tree of order 3 by inserting 10, 20, 30, 40, 50.
5. Construct a Red-Black Tree using 50, 40, 60, 30, 45, 55, 70, 20. Delete 30 and analyze how Red-Black properties are restored.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Correct Tree Construction	20	Tree is constructed correctly with all operations performed accurately	Minor errors in construction	Some operations incorrect	Tree construction mostly incorrect
Understanding of Tree Operations	20	Clearly explains insertion, deletion, search and balancing operations	Explains most operations correctly	Limited understanding	Unable to explain operations
Analysis of Tree Behaviour	30	Correctly analyses balancing, rotations, node split/merge, splaying	Good analysis with minor omissions	Partial analysis	Poor or incorrect analysis
Presentation	20	Well-organized report with accurate observations and conclusion	Good report with minor issues	Basic report with limited observations	Incomplete report
Accuracy of Responses	10	90–100% correct answers	75–89% correct answers	50–74% correct answers	Below 50% correct answers

Faculty In-charge	Course Coordinator	Program Director
Signature: _____ Date: _____	Signature: _____ Date: _____	Signature: _____ Date: _____

Question 1:

SORTING

GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

Red-Black Tree

AVL Tree vs Red-Black
Tree

Splay Tree

Deletion first locates the target node, then carefully reconnects its children so the BST rule still holds. If the node has no children, it is simply removed. If it has one child, that child takes its place. If the node has two children, the tree replaces it with a nearby node that preserves ordering. Learn the full strategy in our [BST deletion](#) guide.

Pseudocode

Enter a number: 81

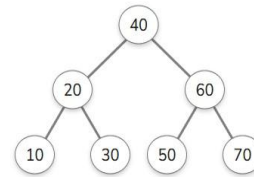
INSERT

DELETE

CLEAR



```
function insert(node, key):  
    if key < node.value  
        if node.left is null:  
            node.left = new Node(key)  
        else:  
            insert(node.left, key)  
    else if key > node.value:  
        if node.right is null:  
            node.right = new Node(key)  
        else:  
            insert(node.right, key)
```

[← Back to Home](#)

Tree Traversals



Controls

Input

Input is level-by-level order.

40,20,10,30,60,50,70

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

Pre

In

Post

Generate Default

Visualize

Start Simulation

30 20 60 40 50 10 70



SORTING

GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

Red-Black Tree

AVL Tree vs Red-Black Tree

Splay Tree

Deletion first locates the target node, then carefully reconnects its children so the BST rule still holds. If the node has no children, it is simply removed. If it has one child, that child takes its place. If the node has two children, the tree replaces it with a nearby node that preserves ordering. Learn the full strategy in our [BST deletion](#) guide.

Pseudocode

Enter a number: 49



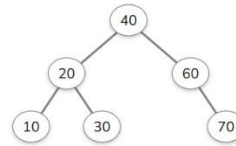
INSERT

DELETE

CLEAR



```
function insert(node, key):  
    if key < node.value:  
        if node.left is null:  
            node.left = new Node(key)  
        else:  
            insert(node.left, key)  
    else if key > node.value:  
        if node.right is null:  
            node.right = new Node(key)  
        else:  
            insert(node.right, key)
```



Question 2:

[← Back to Home](#)

Tree Traversals

?

Controls

Input

Input is level-by-level order.

55,35,75,25,45,65,85

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

Pre

In

Post

Generate Default

Visualize

Start Simulation

55 35 25 45 75 65 85

55

35

75

25

45

65

85

[← Back to Home](#)

Tree Traversals

?

Input

Input is level-by-level order.

55,35,75,25,45,65,85

Enter numbers separated by commas. Use 'null' for empty nodes.

Traversal Order

Pre

In

Post

Generate Default

Visualize

Start Simulation

25 45 35 65 85 75 55

55

35

75

25

45

65

85

© 2025 Saifullah Bin Yusuf. All rights reserved.

🔊

🌐

</>

✉

Question 3:

SEE ALGORITHMS

Articles About Sign in

SORTING

GRAPH

DATA STRUCTURES

Circular Queue
Linked List
Doubly Linked List
Binary Heap
Binary Search Tree
AVL Tree
Red-Black Tree
AVL Tree vs Red-Black Tree
Splay Tree

Pseudocode

Enter a number: 16

function rebalance(node):

updateHeight(node)

nodeBf = balanceFactor(node)

if nodeBf > 1:

if balanceFactor(node.left) >= 0:

rotateRight(node)

else:

rotateLeft(node.left)

rotateRight(node)

if nodeBf < -1:

if balanceFactor(node.right) <= 0:

rotateLeft(node)

else:

rotateRight(node.right)

rotateLeft(node)

AI Summary (5 credits)

Question 4:

SEE ALGORITHMS

Articles About Sign in

SORTING

GRAPH

DATA STRUCTURES

Circular Queue
Linked List
Doubly Linked List
Binary Heap
Binary Search Tree
AVL Tree
Red-Black Tree
AVL Tree vs Red-Black Tree
Splay Tree

node can hold multiple keys and have more than two children. B-trees are widely used in databases and file systems where large blocks of data must be read and written efficiently.

When inserting a new key, it is placed into the appropriate leaf node. If the node overflows by exceeding the maximum number of keys, it **splits** — the median key is pushed up to the parent, while the remaining keys form two child nodes. This process can propagate upward and may even create a new root. This splitting mechanism is essential for keeping the tree balanced, ensuring that all leaves always remain at the same depth as keys are redistributed.

Enter a number: 27

INSERT SEARCH CLEAR

Question 5:

SORTING

GRAPH

DATA STRUCTURES

Circular Queue

Linked List

Doubly Linked List

Binary Heap

Binary Search Tree

AVL Tree

Red-Black Tree

AVL Tree vs Red-Black Tree

Splay Tree

rotations. It maintains balance through four key properties: the root is always black, all leaves (null nodes) are black, **red nodes cannot have red children**, and every path from a node to its descendant leaves contains the same number of black nodes.

2. Check the tree for violations of the above rules.

3. If there's a violation, apply rotations (left or right) and recoloring to restore properties.

4. Repeat until all rules are satisfied from the modified node to the root.

Enter a number: 39



INSERT

CLEAR

